PES UNIVERSITY, Bangalore

Established under Karnataka Act No. 16 of 2013

Department of Computer Science & Engineering

UE24CS341A : Software Engineering

LAB - 05

Name: Kari Bhavitha	SRN: PES1UG25CS821	Section: F

Static Code Analysis – Reflection

1. Which issues were the easiest to fix, and which were hardest?

- ⇨ The easiest issues were formatting, naming, missing docstrings, and unused imports because the tools clearly showed what needed to be changed. The hardest issues were fixing the card encryption and correcting the refund, upgrade charge, and coupon date logic. These needed me to understand how the program worked, not just change the code based on a warning.

2. Did the tools report any false positives?

- ⇨ Most warnings were useful. Pylint showed one warning related to lxml that was caused by how the library works, rather than a problem in my code. Bandit did not warn about the XML parsing risk, so I had to notice that issue separately. This showed me that tools can miss some problems and that I still need to review the code myself.

3. How would you use static analysis in your normal coding work?

- ⇨ I would run Pylint and Flake8 while writing code so I can fix problems early. I would also run Bandit before pushing changes. These checks could be added to GitHub Actions so they run automatically whenever code is submitted. I would keep the tool versions fixed so the results stay consistent.

4. What improvements did you notice?

- ⇨ Pylint improved from 4.43 to 10.00, Flake8 warnings went from 35 to 0, and Bandit findings went from 5 to 0. The code also became safer and worked more correctly. For example, the invoice escapes the username, card encryption uses a fresh random nonce, refunds use the unused days, and secrets are no longer written directly in the code. I also learned that running the program is still important because the tools do not catch every logic problem.